

# CoBit / BitstreamDiffusion — Research Report

Friday 4 September – Tuesday 8 September 2026

## Summary

Four workstreams were carried out over five days and 41 commits: the guidance study was **closed** with a replication verdict; the error taxonomy was completed; the **CE-vs-SM objective** branch was taken through a failed pilot, a post-mortem, a derivation and a multi-seed stability study; and **temporal ordering** was implemented, tested and run to a confirmed result.

**Headline: no intervention tested improved GSM8K accuracy.** Temporal ordering is neutral at best and harmful at worst. The objective replacement could not be evaluated for task performance at any budget this environment permits. Both are clean, well-powered negatives rather than inconclusive runs. **The dominant finding is infrastructural:** every training run here diverges before  $\sim 12k$  steps, while the same code trained 500k steps cleanly in the collaborator’s environment.

## 1 Research table

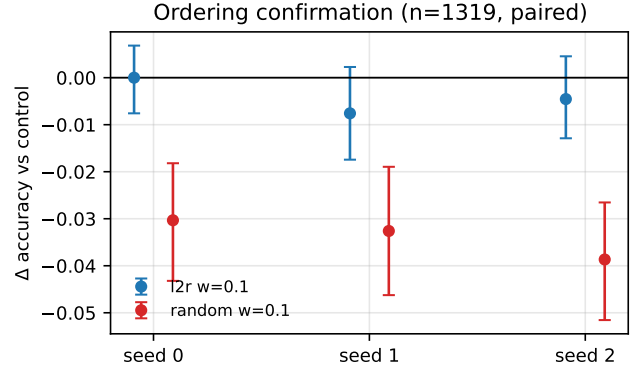
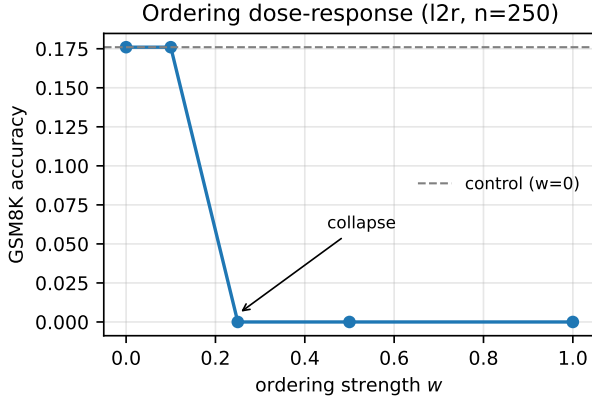
| Experiment          | Control        | Intervention             | Steps | Seeds    | GPU-h | Acc.          | $\Delta$ / CI               | Verdict                     |
|---------------------|----------------|--------------------------|-------|----------|-------|---------------|-----------------------------|-----------------------------|
| A1 Ordering screen  | $w=0$          | l2r $w=0.1$              | 256   | 1        | 0.65  | 0.1760        | +0.0000<br>[−0.020, +0.020] | no effect                   |
| A1                  | $w=0$          | l2r $w \geq 0.25$        | 256   | 1        | —     | 0.0000        | −0.176<br>[−0.224, −0.132]  | $\neg$ collapse             |
| A1                  | $w=0$          | l2r $w=0.25$             | 512   | 1        | —     | 0.0000        | −0.184<br>[−0.232, −0.136]  | $\neg$ notstep – st         |
| A2 Ordering confirm | $w=0$ (0.1387) | <b>l2r</b> $w=0.1$       | 256   | <b>3</b> | 1.55  | 0.1347        | −0.0040, 0/3 sig.           | <b>no effect</b>            |
| A2                  | $w=0$          | <b>random</b><br>$w=0.1$ | 256   | <b>3</b> | —     | 0.1049        | −0.0339,<br>$p < 10^{-4}$   | <b>3/3</b> $\neg$ harmful   |
| B1 Objective        | sm @5k         | ce @5k                   | 5k    | 1        | 0.16  | 0.000 / 0.000 | both 250/250 T1             | $\neg$ unmeasura            |
| B2 Lift-off         | —              | sm/ce @10k               | 10k   | 1        | 0.06  | 0.0000        | —                           | still zero at 1             |
| B3 Stability        | binary_sm      | binary_ce                | 20k   | 3+3      | 36.5  | n/a           | rank-sum $p=0.05$           | CE $1.80 \times$ long       |
| C Prod-era diag.    | HEAD           | <b>036a2b5</b>           | 20k   | 1        | 6.50  | n/a           | broke @11,278               | our code exon               |
| Anchor              | —              | production<br>500k       | 500k  | 1        | —     | <b>0.1640</b> | —                           | ated<br>eval pipel<br>sound |

Ordering/objective spend this window  $\approx 2.4$  GPU-h; whole-study total  $\approx 156$  GPU-h. Accuracy on the 1319-problem GSM8K test set (screens use a fixed 250-problem subset). All comparisons paired, bootstrap over problems.

## 2 Temporal ordering (Branch A)

**Definition.** Per-token time  $t_j(t) = \text{clip}(t(1+w) - w u_j, 0, 1)$ , with  $u_j \in [0, 1]$  a denoising *priority* ( $u=1$  denoises first) defined over the generated **suffix only**. Prompt positions keep the global  $\sigma$ . At  $w=0$  every token gets the global time, i.e. today’s model — verified bit-identically.

**The intervention is causally active**, checked before any GPU spend: at  $w=0.5$  the per-position  $\sigma$  spread is  $192 \times$  with the first suffix token cleanest; at  $w=1.0$  it is  $11,247 \times$ ; the prompt stays pinned at the global value in all cases.

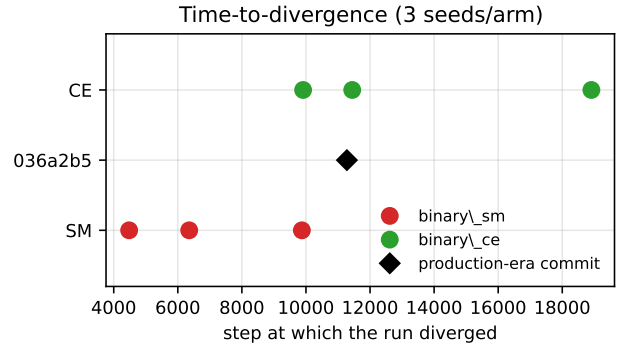
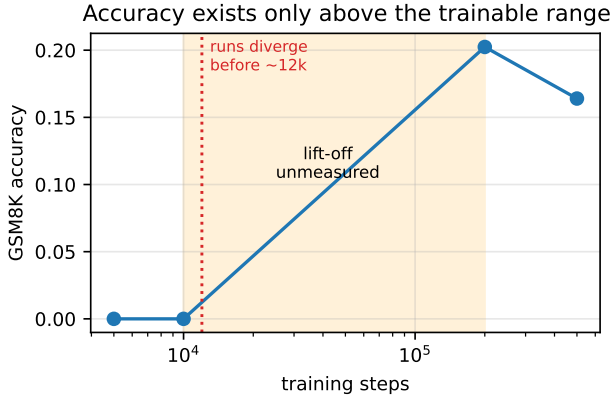


**Result.** At the only strength the model survives ( $w=0.1$ ) ordering has **no effect** ( $-0.0040$ , 0/3 seeds significant, CIs  $\pm 0.008$ – $0.017$ ). Random ordering is **reliably harmful** ( $-0.0339$ , 3/3 seeds,  $p < 10^{-4}$ ). Beyond  $w=0.25$  the model collapses to zero, and doubling the trajectory to 512 steps does not rescue it.

**Interpretation.** The l2r/random asymmetry shows the ordering is doing something real — direction matters — but nothing helps. The model was *trained* with uniform  $\sigma$ , so per-position  $\sigma$  is out-of-distribution: it tolerates  $\sim 2.9\times$  spread with no gain and breaks at  $\sim 9\times$ . This does **not** falsify ordering as a *training* objective, which is untestable here.

### 3 Objective replacement (Branch B)

The meaningful comparison is `binary_sm` vs `binary_ce`. The  $\sigma$ -weighting was **derived rather than inherited**: matching Bayes-risk scale gives a factor spanning only 0.250–0.361 across four decades of  $\sigma$  ( $0.82$ – $1.18\times$  anchored at  $\sigma_d$ ), while gradient-scale matching is self-defeating because it reintroduces the  $D(1-D)$  factor under test. Conclusion: **no bespoke CE weighting is justified**; both arms use EDM weighting, clamp off.



**Task performance: not answerable here.** At a matched 5k steps both arms score **exactly 0/250**, with 250/250 non-executing programs and invalid-token rates differing by 0.001. At 10k both are still 0. The production anchor scores 0.164 under identical settings, so the evaluation is sound — the models are simply nowhere near converged. Accuracy lift-off lies somewhere in  $[10k, 200k]$  and is **unmeasured**.

**Stability: CE lasts  $1.80\times$  longer.** Break steps — SM {4481, 6356, 9872}, CE {9911, 11444, 18909} — perfect rank separation, exact one-sided rank-sum  $p = 1/20 = 0.05$ , the floor of a  $3\times 3$  design. *But* the production-era commit broke at 11,278, inside the CE band, so break timing moves more across a numerics-neutral code change than the effect being measured.

### 4 The infrastructural finding

Seven runs across two code versions diverged before  $\sim 12k$  steps, including the production-era commit 036a2b5 itself. The same code ran 500k steps with *zero* excursions in the collaborator’s environment. Ruled out: corpus

( $\sigma_{\text{data}}$  fingerprint matches to  $\sim 8$  s.f.), data config (identical on all 22 keys), and our post-June code changes (fp32 forward+loss+backward **bit-identical** to the production commit).

Two independent proofs the environments differ: the historical code imports `mauve` at module scope (their env had it, ours does not); and `diffusion/discrete/losses.py` has used PEP 604 in a signature since the initial release, which *cannot import on Python 3.9* — so production ran on  $\geq 3.10$  while we run 3.9. The production directory is permission-denied, so their working tree cannot be inspected.

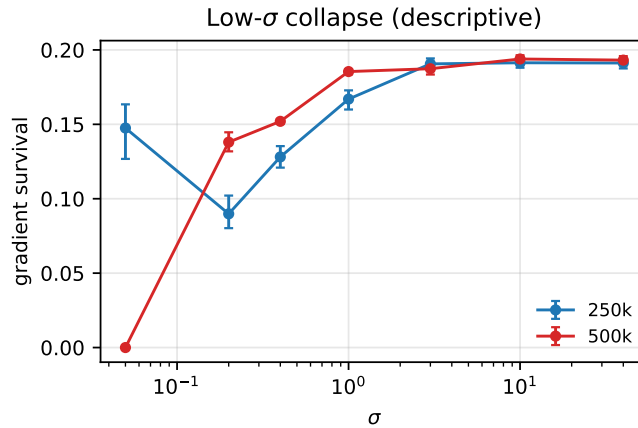
**Divergence signature:** an abrupt  $\sim 400\times$  jump inside one 20-step interval, with **no precursor** in gradient norm, logit magnitude, update RMS or gradient survival; Adam’s `exp_avg_sq` is *declining* beforehand, then jumps  $33\times$ . A single-batch loss spike poisons the optimiser state.

## 5 Hypotheses falsified

| Hypothesis                                           | Outcome                                                   |
|------------------------------------------------------|-----------------------------------------------------------|
| Ordering improves generation                         | $\neg$ <i>Falsified</i> (well – powerednull; randomworse) |
| CE’s advantage appears as retained gradient survival | $\neg$ <i>Falsified</i> – – allarms $\sim 0.18$ pre-break |
| Low- $\sigma$ weight amplifier causes divergence     | $\neg$ <i>Falsified</i> bytheclamp $2\times 2$            |
| The clamp caused the early break                     | $\neg$ <i>Falsified</i> – – all6clamp – offrunsdiverged   |
| Global SM gradient starvation                        | $\neg$ <i>Withdrawn</i> – – survivalstable0.13 – –0.19    |
| Our post-June code caused the divergence             | $\neg$ <i>Falsified</i> – – prod – eracommitdivergestoo   |
| CE is stable                                         | $\neg$ <i>Falsified</i> – – the5kconclusionwasanartefact  |

## 6 Low- $\sigma$ saturation — kept separate

Robust and well measured ( $n=128$ , 2 noise seeds, fully paired): gradient survival at  $\sigma=0.05$  falls  $0.1475 \rightarrow$  **exactly 0** between 250k and 500k, with 100.00% of free bits saturated in *every* evaluation. Everywhere else training *improves* survival. **This is descriptive and causally unlinked** to the divergences, which occur at 4.5k–19k — nowhere near a 250k model.



## 7 Limitations and confounds

1. **Environment-limited.** All stability numbers are specific to this Python 3.9 stack. CE’s  $1.80\times$  may be an artefact of a broken toolchain rather than a property of the objectives.
2. **Ordering was tested at decoding time only**, on a model trained with uniform  $\sigma$ . Ordering-as-training remains untested.
3. **Branch B is budget-blocked, not answered.** Divergence before 12k and zero accuracy below 10k leave *no window*.
4.  $n=3$  seeds;  $p=0.05$  is the design floor.
5. The production working tree is unreadable, so uncommitted differences cannot be excluded.

6. **Two of my own bugs invalidated  $\sim 1.2$  GPU-h:** an over-strict guidance guard failed 9/9 cells, and the ordering control returned probabilities from  $\sigma_{N-1}$  instead of a final denoise, scoring 0.000 against the baseline’s 0.164. The latter was caught only because a production anchor was included — without it, seven cells of *confidently wrong* nulls would have been reported.

## 8 Recommendation

1. **Close temporal ordering as a decoding intervention.** The result is well powered and negative. Ordering-as-training is a separate, untested question.
2. **Fix the environment before any further objective work.** A Python 3.10 environment (sedd310, torch 2.8.0+cu128) has been built; the single-variable validation run — seed 42, the one that broke at 6,356 — is ready.  $\sim 10$  GPU-h.
3. **Do not commit 300+ GPU-h** to a from-scratch CE-vs-SM comparison on the strength of a  $p=0.05$  stability difference measured in a suspect environment. If a performance answer is needed sooner, branch both arms from the healthy 500k checkpoint ( $\sim 25\text{--}73$  GPU-h), accepting that it answers “does switching to CE help this model” rather than “which objective trains better”.